HSE, VK

28/01/2026

Singular Value Decomposition in Recommender Systems.

Alexander Tarakanov

Outline

Rank of the Matrix

Let $A \in \mathbb{R}^{m \times n}$ be a matrix.
Rank of A is defined as:

$$\operatorname{rank}(A) = \dim\big(\operatorname{Im}(A)\big) = \text{number of linearly independent columns (or rows).}$$

Equivalently:

$$\operatorname{rank}(A) = \max\{r : A = BC,\ B \in \mathbb{R}^{m \times r},\ C \in \mathbb{R}^{r \times n}\}.$$

Matrix factorisation viewpoint:

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^\top$$

where $r = \operatorname{rank}(A)$ and each term is a rank-one matrix.

Key idea: Low-rank matrices admit compact representations and capture hidden structure.

Singular Value Decomposition

For any matrix $A \in \mathbb{R}^{m \times n}$ there exists a decomposition:

$$A = U \Sigma V^\top$$

where:

- $U \in \mathbb{R}^{m \times m}$, $U^\top U = I$
- $V \in \mathbb{R}^{n \times n}$, $V^\top V = I$
- $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_{\min(m,n)})$

with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

Rank-k truncation:

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top$$

Optimality (Eckart–Young–Mirsky Theorem):

$$A_k = \arg \min_{\mathrm{rank}(B) \leq k} \|A - B\|_F$$

Thus, truncated SVD gives the *best* low-rank approximation.

Power Method

Goal: compute the dominant eigenpair of $A \in \mathbb{R}^{n \times n}$.
Assume:

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$$

Iteration:

$$x_{k+1} = \frac{Ax_k}{\|Ax_k\|}$$

starting from random x_0.

Then:

$$x_k \to \pm v_1, \quad \lambda_1 \approx x_k^\top A x_k$$

For SVD: apply the power method to $A^\top A$ or $AA^\top$.

Convergence Estimate

Let $x_0 = \sum_i c_i v_i$, where v_i are eigenvectors of A.
Then:

$$x_k \propto \sum_i c_i \lambda_i^k v_i$$

Dominant error term:

$$\|x_k - v_1\| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right)$$

Key factor: spectral gap

$$\frac{|\lambda_2|}{|\lambda_1|}$$

Smaller ratio $\Rightarrow$ faster convergence.

Convergence Estimate (Interpretation)

- Fast convergence if eigenvalues are well-separated
- Slow convergence if spectrum is flat
- Power method fails when $\lambda_1 \approx \lambda_2$

Practical consequence:

- Works well for strongly low-rank matrices
- Less efficient for noisy data

Computing the First k Eigenvectors

The power method computes only the dominant eigenvector v_1. To compute the first k eigenvectors, we proceed iteratively.

Step 1: Compute (λ_1, v_1) using the power method.

Step 2 (Deflation): Remove the contribution of v_1 from the matrix:

$$A^{(1)} = A - \lambda_1 v_1 v_1^\top$$

Step 3: Apply the power method to $A^{(1)}$ to obtain (λ_2, v_2).

Repeat the procedure to obtain

$$(\lambda_1, v_1), (\lambda_2, v_2), \ldots, (\lambda_k, v_k)$$

Remark: Orthogonalization is required for numerical stability.

Power Method Example

Consider diagonal matrix:

$$A = \mathrm{diag}(1,\ \rho)$$

Cases:

- $\rho = 0.1$: fast convergence
- $\rho = 0.9$: slow convergence
- $\rho = 0.99$: extremely slow

Error decay:

$$\|x_k - v_1\| \sim \rho^k$$

Illustrates importance of eigenvalue contrast.

Power Method for Smallest Eigenvalues

To compute smallest eigenvalue $\lambda_{\min}$:

Inverse iteration:

$$x_{k+1} = \frac{A^{-1}x_k}{\|A^{-1}x_k\|}$$

Equivalent to power method on A^{-1}.

Shifted version:

$$(A - \mu I)^{-1}$$

Used in practice to target specific spectral regions.

Outline

Issues with SVD

User–item matrix $R \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}$:

- Extremely sparse
- Missing entries $\neq 0$
- Classical SVD requires full matrix

Naive SVD fails:

$$\text{zero-filling} \Rightarrow \text{biased solution}$$

Solution: formulate SVD as an optimisation problem over observed entries.

SVD as Optimisation Problem

We seek a low-rank approximation:

$$R \approx PQ^\top$$

where:

$$P \in \mathbb{R}^{|\mathcal{U}| \times k}, \quad Q \in \mathbb{R}^{|\mathcal{I}| \times k}$$

Objective:

$$\min_{P,Q} \sum_{(u,i) \in \Omega} \left(R_{ui} - p_u^\top q_i\right)^2$$

This is equivalent to truncated SVD for fully observed R.

Gradient Descent

For objective $f(\theta)$ with L-Lipschitz gradient:

$$\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k)$$

If $0 < \eta < \frac{2}{L}$, then:

$$f(\theta_k) - f^\star = \mathcal{O}\left(\frac{1}{k}\right)$$

For quadratic problems: linear convergence.
Used to optimise matrix factorisation objectives.

Gradient Descent Issues and Problems

- Full gradient cost: $\mathcal{O}(|\Omega|k)$
- Requires multiple passes over data
- Sensitive to learning rate

Non-convexity:

$$f(P, Q) \text{ is not jointly convex}$$

But:

- All local minima are global (under mild assumptions)
- Saddle points dominate in practice

Stochastic Gradient Descent

Instead of full gradient:

$$\nabla f(\theta) \approx \nabla f_{ui}(\theta)$$

Update rule:

$$\theta_{k+1} = \theta_k - \eta_k \nabla f_{ui}(\theta_k)$$

In ML terminology:

- online learning
- mini-batch optimisation

Learning Rate Decay for Convergence

To ensure convergence:

$$\sum_{k=1}^{\infty} \eta_k = \infty$$

Example:

$$\eta_k = \frac{\eta_0}{k}$$

Guarantees convergence to stationary point.

Learning Rate Decay for Finite Variance

To control gradient noise:

$$\sum_{k=1}^{\infty} \eta_k^2 < \infty$$

Typical choice:

$$\eta_k = \frac{\eta_0}{\sqrt{k}}$$

Trade-off:

- Fast initial learning
- Stable asymptotic behaviour

SGD for SVD

Regularised objective:

$$\min_{P,Q} \sum_{(u,i)\in\Omega} (R_{ui} - p_u^\top q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$$

SGD updates:

$$p_u \leftarrow p_u + \eta(e_{ui} q_i - \lambda p_u)$$

$$q_i \leftarrow q_i + \eta(e_{ui} p_u - \lambda q_i)$$

where:

$$e_{ui} = R_{ui} - p_u^\top q_i$$

This is the standard SVD model used in recommender systems.

Outline

Outline

Graph Laplacian Operator

A point cloud $x_1, \ldots, x_N$ in $\mathbb{R}^d$.

A fully connected graph with weights:

$$W_{ij} = \exp\left(-\gamma \|x_j - x_i\|^2 \right)$$

Graph Laplacian operator:

$$L = D - W,$$

where D is a diagonal matrix with

$$D_{ii} = \sum_{j=1}^{N} W_{ij}.$$

For any function f defined on the graph, the following inequality holds:

$$L(f, f) \geq 0.$$

$$f^\top L f = \sum_{i,j} L_{ij} f_i f_j = \sum_{i,j} (D_{ij} - W_{ij}) f_i f_j$$

Eigenfunctions of the Graph Laplacian

A fact from linear algebra: for a graph V with N vertices, there exist functions $\phi_1, \ldots, \phi_N$ such that the following conditions hold:

$$\langle \phi_i, \phi_j \rangle = \sum_{v \in V} \phi_i(v)\phi_j(v) = \begin{cases} 1, & i = j, \\ 0, & i \neq j \end{cases}$$

$$\sum_{a,b=1}^{N} L_{ab}\phi_i(v_a)\phi_j(v_b) = \begin{cases} \lambda_i, & i = j, \\ 0, & i \neq j \end{cases}$$

Here,

$$0 \leq \lambda_1 \leq \cdots \leq \lambda_N$$

are the eigenvalues, and $\phi_1, \ldots, \phi_N$ are the corresponding eigenfunctions.

Moreover, any function f on the graph can be uniquely represented as:

$$f = \sum_{k=1}^{N} \xi_k \phi_k,$$

where $\xi_1, \ldots, \xi_N$ are expansion coefficients.

Properties of Graph Laplacian Eigenvectors

The constant function $\phi_1(v) = 1$ is an eigenvector with zero eigenvalue:

$$\lambda_1 \phi_1^\top \phi_1 = \phi_1^\top L \phi_1 = \frac{1}{2} \sum_{ij} W_{ij} \big(\phi_1(v_j) - \phi_1(v_i)\big)^2 = 0$$

If the graph V can be decomposed into two disconnected components V_1 and V_2, then the indicator functions $\chi_1(v)$ and $\chi_2(v)$ are eigenfunctions with zero eigenvalues:

$$\chi_a(v) = \begin{cases} 1, & v \in V_a, \\ 0, & \text{otherwise} \end{cases}$$

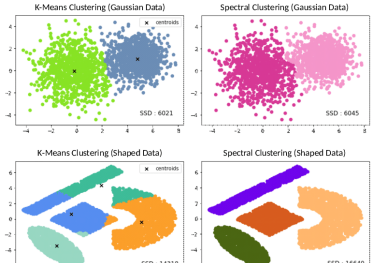
$$\chi_a^\top L \chi_a = \frac{1}{2} \sum_{ij} W_{ij} \big(\chi_a(v_j) - \chi_a(v_i)\big)^2 = 0$$

The case of two disconnected components generalizes naturally to an arbitrary number of components $V_1, \ldots, V_m$.

Spectral Clustering

The classical spectral clustering algorithm can be summarized as follows:

1. Compute the weight matrix
$$w_{ij} = \exp\left(-\gamma d(x_i, x_j)^2\right)$$
2. Compute the Laplacian matrix L
3. Compute the first few eigenfunctions of L:
$$\phi_1, \ldots, \phi_m$$
4. Construct a feature vector for each vertex:
$$[\phi_1(v), \ldots, \phi_m(v)]^\top$$
5. Apply the K-means clustering algorithm



Semi-Supervised Learning

Consider an image where X denotes the set of pixels (tabular data with RGB values). A small subset of pixels S is labeled, with labels taking values in $\{-1, 1\}$ (e.g., object vs background).

How can we infer the labels of the remaining pixels?

This can be formulated as a regularized regression problem:

$$\mathcal{L}(c) = \sum_{v \in S} \left(y(v) - \sum_{j=1}^{m} c_j \phi_j(v) \right)^2 + \lambda \|c\|_2^2$$

Connection Between Continuous and Discrete Laplacians

The Laplace–Beltrami operator can be approximated by the following integral:

$$-\Delta_{\mathcal{M}} f(p) \approx L_t(f)(p) = \frac{1}{t(4\pi t)^{\dim(\mathcal{M})/2}} \int_{\mathcal{M}} \exp\left(-\frac{|x-p|^2}{4t}\right)\big(f(x)-f(p)\big)\sqrt{|g(x)|}\,dx$$

This approximation becomes exact as $t \to 0$.

The second key idea of the proof is to observe that the action of the discrete Laplacian corresponds to a Monte Carlo approximation of $L_t(f)$:

$$\frac{N}{\mathrm{vol}(\mathcal{M})} L_t(f)(x_i) \approx \sum_{ij=1}^{N} w_{ij}(t)\big(f(x_j)-f(x_i)\big) = [(D(t)-W(t))f]_i$$

It can be shown that with an appropriate choice

$$t(N) = t_0 N^{-2/(n+\epsilon)},$$

the eigenfunctions of the graph Laplacian converge to the eigenfunctions of the continuous Laplace–Beltrami operator.

User and Item Embeddings via Graph Laplacian

Consider a user–item interaction graph:

$$G = (\mathcal{U} \cup \mathcal{I}, E)$$

where vertices correspond to users and items, and edges represent observed interactions.

Define a weighted adjacency matrix W and the graph Laplacian:

$$L = D - W \quad \text{or} \quad L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}.$$

Compute the first k eigenvectors of the Laplacian:

$$L\phi_\ell = \lambda_\ell \phi_\ell, \quad \ell = 1, \ldots, k.$$

Spectral embeddings:

- User embedding for $u \in \mathcal{U}$:

$$z_u = \big(\phi_1(u), \ldots, \phi_k(u)\big)^\top$$

- Item embedding for $i \in \mathcal{I}$:

$$z_i = \big(\phi_1(i), \ldots, \phi_k(i)\big)^\top$$

Interpretation: Low-frequency eigenvectors encode smooth, community-aware representations over the interaction graph.

Outline

Concluding Remarks

- Singular Value Decomposition provides the **optimal low-rank representation** of data matrices and serves as the mathematical foundation of matrix factorization methods in recommender systems.
- SGD and its variants enable **scalable computation** of leading eigenvectors and singular vectors for large, sparse matrices.
- Graph Laplacian eigenvectors generalize SVD to non-Euclidean data and yield **spectral embeddings** for users and items.
- Spectral methods reveal community structure but rely on **global smoothness assumptions**, which may limit expressiveness.
- Modern graph-based models (GNNs, sheaf neural networks) can be viewed as **learnable, structure-aware generalizations** of spectral and SVD-based approaches.

$$\textit{SVD} \rightarrow \textit{Spectral Methods} \rightarrow \textit{Graph Neural Networks}$$